

Lab Work — Algorithmics and Dynamic Data Structures (ADDS)

Using Set Theory and Dynamic Data Structures to Manage Linguistic Structures

1. Introduction

Set theory constitutes one of the most fundamental branches of mathematics, providing a rigorous framework for reasoning about collections of objects. Formally, a set is a well-defined, unordered collection of distinct elements. Sets are conventionally denoted by uppercase letters (e.g., A , B , S), while their elements are denoted by lowercase letters. Membership of an element x in a set A is written $x \in A$, and non-membership as $x \notin A$.

A set can be defined by extension — by explicitly listing its elements, e.g., $A = \{\text{the, a, is}\}$ — or by intension — by a property its elements satisfy, e.g., $A = \{x \mid x \text{ is a stopword}\}$. The cardinality of a set, denoted $|A|$, refers to the number of its elements. A set containing no elements is called the empty set, denoted $\emptyset$.

Fundamental operations. Given two sets A and B defined over a common universe U , the principal operations are:

- **Union:** $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$ — the set of all elements belonging to at least one of the two sets.
- **Intersection:** $A \cap B = \{x \mid x \in A \text{ and } x \in B\}$ — the set of elements common to both sets.
- **Difference:** $A \setminus B = \{x \mid x \in A \text{ and } x \notin B\}$ — the elements of A that are not present in B .
- **Subset:** $A \subseteq B$ — if every element of A is also an element of B .

Relevance to linguistic data. Set theory provides a natural framework for modelling linguistic phenomena. A document can be represented as a set of distinct terms, and linguistic categories — such as stopwords or named entities — as subsets of a broader lexicon. When implemented through dynamic data structures, set operations can be performed efficiently on large and evolving linguistic datasets, making set theory both a theoretical and practical tool in text processing.

2. Objectives

The main objective of this lab work is to leverage dynamic data structures — including dynamic arrays, linked lists, stacks, queues, and trees — to represent and manipulate text paragraphs extracted from files, and to implement set-theoretic operations on these representations. The targeted operations are:

- **Union:** merging two structural representations to form a combined vocabulary or document.
- **Intersection:** identifying shared terms or patterns between two text representations.
- **Difference:** extracting terms present in one representation but absent from another.

3. Required Work

Phase 1 — Representation of Text Paragraphs

- Propose and justify an appropriate dynamic data structure for representing and managing text paragraphs.
- Define the necessary abstract machines associated with the proposed dynamic data structure.
- Read and pre-process one or more text files to extract their constituent paragraphs.
- Populate the proposed data structure with the extracted paragraphs from each file independently.

Phase 2 — Implementation of Set Operations

- Implement the functions and procedures required to perform union, intersection, and difference operations on the proposed data structure.
- The set operations shall be applicable either to paragraphs within the same file or to paragraphs across different files; the user shall be able to select the desired option at runtime.

Phase 3 — Testing and Evaluation

- **Step 1:** Validate the correctness and effectiveness of the implemented set operations on user-selected text files.

- **Step 2:** Evaluate the proposed solution on blind test files provided on the day of the presentation.

4. Deliverables

Students are required to submit the following:

- A complete and well-structured implementation of the proposed dynamic data structure in the C programming language.
- Implementation of the abstract machines associated with the proposed data structure.
- Implementation of the functions required to read, pre-process, and represent text paragraphs as sets.
- Implementation of the set operations: union, intersection, and difference.
- A user-friendly interface demonstrating the testing and validation of the proposed solution.
- A concise slide presentation summarising the proposed data structure, abstract machines, and implemented functions.

5. Instructions and Constraints

- The entire implementation must be written in **C language**.
- The code must be clearly organised, well structured, and sufficiently commented.
- **Academic integrity:** plagiarism detection tools will be systematically applied. Any submission found to be plagiarised will receive a grade of **0** without exception.
- **Submission deadline:** 10/05/2026. Late submissions will be penalised at a rate of **-2 points per day**.